



Canali

Lezione
23
Corso 1



Modelli di concorrenza

Struttura dati condivisa: ci si **sincronizza** per **comunicare**

Scambio di messaggi: si **comunica** per **sincronizzarsi**.



Condivisione di messaggi

- In alternativa alla condivisione dello stato, Rust offre un meccanismo di comunicazione e sincronizzazione tra thread basato sulla condivisione di messaggi
 - La funzione `std::sync::mpsc::channel<T>()` restituisce una coppia ordinata (tupla) formata da una `struct Sender<T>` ed una `struct Receiver<T>`
 - Tutti i dati inviati tramite il metodo `send(...)` della prima possono essere consumati attraverso il metodo `recv()` della seconda, nello stesso ordine in cui sono stati inviati
 - Il metodo `send(...)` offre la garanzia che chi lo invoca non sarà bloccato (ovvero il canale di comunicazione ha una capacità infinita di memorizzazione temporanea dei messaggi)
 - Il metodo `recv()` si blocca senza consumare cicli macchina in attesa di un messaggio o della terminazione dell'oggetto `Sender` e di tutti i suoi eventuali cloni
- Il dato trasmesso deve essere di tipo **T** e deve godere del tratto **Send**
- L'implementazione fornita gode della proprietà **multiple producer - single consumer** ovvero permette di creare più cloni dell'oggetto *sender*
 - Mentre obbliga ad avere una singola copia dell'oggetto *receiver*

Condivisione di messaggi

- In questo modello di comunicazione, **il singolo dato prodotto da un thread viene ceduto al canale e da questo al thread ricevente che diventa il possessore finale del valore**
 - Questa operazione agisce al tempo stesso da **sincronizzazione** (la ricezione è necessariamente successiva all'invio, ossia non posso leggere fino a che qualcuno non abbia scritto) e da **comunicazione** (il dato passato rappresenta l'unità di messaggio)
- Un numero arbitrario di messaggi può essere scambiato sul canale
 - A condizione che il ricevitore sia attivo
 - Se il ricevitore viene deallocato, eventuali tentativi di invio falliscono con la generazione di un valore di tipo **SendError<T>**
 - Se tutti i trasmettitori vengono deallocati, tentativi di lettura sul ricevitore falliscono con la generazione di un valore di tipo **RecvError**

Scambio di messaggi

- RUST ci mette a disposizione una primitiva elementare che si chiama `channel`
- Un channel è un tubo a 2 estremità: quella di ingresso e quella di uscita. All'interno del channel possono essere presenti dei messaggi.
- Chi conosce l'estremità di ingresso può produrre un dato e cederlo al canale. Il dato viene prodotto e affidato al canale. Sta nel canale fino a quando qualcun altro, nell'estremità di uscita non cerca di leggerlo. Chi cerca di leggerlo acquisisce il possesso di quel dato e ci fa quello che vuole
- Il ricevitore legge il valore solo se un trasmettitore lo ha prodotto.
- Contemporaneamente ottengo 2 informazioni:
 - So che l'altro ha fatto un certo lavoro
 - Ottengo un dato corrispondente al lavoro svolto.

Operativamente

- Preparo il channel prima di creare i 2 thread che hanno bisogno di parlarsi
- Do l'estremità di ingresso al thread sender
- Do l'estremità di uscita al thread receiver
- Il canale conserva l'ordine con cui sono stati scritti i messaggi.
- Chi legge, legge alla velocità che può. Finchè non c'e' nulla nel tubo, rimane bloccato, senza consumare CPU.
- Non ho bisogno di usare la condition variable, perche' se il canale è vuoto, l'operazione di lettura mi sospende automaticamente. Appena qualcuno inietta qualcosa nel canale, se io sono in attesa, mi sveglio e lo prendo. Se non sono in attesa, sta nel canale.
- Vado nel canale, prendo il dato e poi torno nel canale a vedere se c'è un altro dato.

- Il Channel ci offre un modo per far sapere al receiver che non potrà più ricevere nulla perche' non ci sono più sender: non appena tutti i sender vanno via, il receiver esce con un errore che dice che non ci sono più sender e quindi può smettere, senza aver bisogno di sapere a priori di quanti pezzi dovrà aspettare.
- Parallelamente, anche il sender riceve una indicazione. Se l'unico receiver andasse via (ossia viene distrutto e se ne fa il drop), un tentativo di invio fallisce poiché il channel sa che non c'è più la possibilità di ricevere il dato.

- Il producer crea un dato che sta possedendo, quando lo vogliono comunicare al ricevitore, lo cedono al canale. Questo permette al producer di non avere nessuna idea dell'identità del consumer. L'unica cosa che conosce il producer è il canale. Il canale lo renderà disponibile appena possibile.
- Il canale non è limitato. Io posso mettere nel canale quanti messaggi voglio (unbounded). Se quell'altro non sta leggendo perché è occupato, i dati stanno nel canale. E' come se fosse un tubo che si allunga per contenere i dati.
- Appena il ricevitore chiede di leggere, legge il primo (quello più vecchio).
- Il canale mantiene l'ordine di ricezione. I messaggi vengono letti nell'ordine con cui sono stati inseriti.

- Quando l'ultimo sender esce di scena e il canale diventa vuoto. il receiver riceve un error (RecvError). Questo permette di capire, da parte del receiver, che lui deve finire.
- Non basta che siano morti tutti i sender, se ho ancora dei messaggi nel canale, questi vengono svuotati.
- Il canale mi garantisce che tutto quello che lui ha preso in carico verrà smaltito e lo consegnerà (ammesso che vi sia un receiver).
- Parallelemente il sender sa che tutto quello che mette nel canale finirà a destinazione, a patto che ci sia una destinazione. Se per qualche motivo la destinazione cessa di esistere, perché viene fatto il drop, ciò che c'è già nel canale, va perduto (perché nessuno è più in grado di leggerlo) e non posso più aggiungere nulla. Ogni eventuale tentativo di aggiunta ulteriore da origine ad un SendError che mi informa che non c'è più un receiver

```
use std::sync::mpsc::{channel};
use std::thread;
```

```
fn main() {
    let mut handles = vec![];
    let (tx, rx) = channel();
```

Channel() restituisce una tupla:
nel campo 0 il sender e nel campo 1 il receiver

channel1.rs

```
    for _ in 0..3 {
        let tx = tx.clone();
        // cloned tx dropped within thread
        handles.push(thread::spawn(move || tx.send("ok").unwrap()));
    }
```

Prende il possesso delle variabili libere

```
// Drop the last sender to stop `rx` waiting for message.
// The program will not complete if we comment this out.
// **All** `tx` needs to be dropped for `rx` to have `Err`.
```

```
drop(tx);
```

Quanti trasmettitori ci sono in questo punto di esecuzione ?

```
while let Ok(msg) = rx.recv() {
    println!("{}", msg);
}
for handle in handles {
    handle.join().expect("Error");
}
}
```

```

use std::sync::mpsc::{Sender, channel, Receiver};
use std::thread;
struct SharedMsg{
    tx:Sender<String>,
    rx:Receiver<String>
}
impl SharedMsg{
    pub fn new()->Self{
        //Creazione del canale
        let (tx,rx)=channel::<String>();
        SharedMsg { tx, rx }
    }
}
fn main() {
    let mut handles = vec![];
    let shared=SharedMsg::new();
    let tx1=shared.tx.clone();
    handles.push(thread::spawn(move|| {
        if tx1.send("Ciao".to_string()).is_err()==true { println!("Errore nell'invio del messaggio"); }
    }));
    let tx2=shared.tx.clone();
    handles.push(thread::spawn(move|| {
        if tx2.send("Come stai?".to_string()).is_err()==true{ println!("Errore nell'invio del messaggio"); }
    }));
    // Occorre chiudere il canale,
    // Senza questo il receiver continuerebbe a rimanere in attesa di messaggi
    drop(shared.tx);
    while let Ok(msg)=shared.rx.recv(){
        println!("Messaggio ricevuto:");
        println!("{}",msg);
    }
    for handle in handles {
        handle.join().expect("Error");
    }
}

```



```
use std::sync::mpsc::{channel};
use std::thread;
use std::sync::{Arc, Mutex};
use std::time::Duration;
```

```
#[derive(Debug)]
pub struct Task {
    id: usize,
    testo: String,
}
```

```
fn main() {
    let mut handles = vec![];
    let (task_sender, task_receiver) = channel::<Task>(); // sintassi per specificare il tipo trasmesso

    // Per tener traccia di quanti task sono stati effettuati
    let result_counter = Arc::new(Mutex::new(0));

    for i in 0..2 {
        let tx = task_sender.clone();
        handles.push(thread::spawn(move || {
            for j in 0..1_000 {
                let task = Task {
                    id: i * 5 + j,
                    testo: format!("Task {} from Producer {}", i * 5 + j, i),
                };
                tx.send(task).unwrap(); //invio il dato
                thread::sleep(Duration::from_nanos(1));
            }
        })));
    }
}
```



segue

```

let result_counter_t = Arc::clone(&result_counter);
handles.push(thread::spawn(move || {
loop {
    //Per ricevere il dato e poi stamparlo mi metto in loop sul receiver
    match task_receiver.recv() {
        Ok(task) => {
            //println!("Consumer received task: {:?}", task);
            thread::sleep(Duration::from_nanos(1));

// Incremento il mutex per tener traccia delle task effettuate
        {
            let mut counter = result_counter_t.lock().unwrap();
            *counter += 1;
        }
        Err(_) => break, //Esco dal loop quando il channel si chiude
    }
}
}));

// Chiudo il canale con il drop del sender. Necessario per terminare il receiver
drop(task_sender);

for handle in handles {
    handle.join().expect("Error");
}

// Stampo il numero di task facendo lock sul mutex
let counter = result_counter.lock().unwrap();
println!("Total tasks processed: {}", counter);
}

```



Canali sincroni

- La funzione `std::sync::mpsc::sync_channel<T>(bound: usize)` restituisce invece una coppia di valori di tipo `(SyncSender<T>, Receiver<T>)`
 - A differenza di un canale semplice, questo è limitato: se il numero di messaggi giacenti nel canale raggiunge il limite definito (`bound`) le invocazioni del metodo `send(...)` diventano **bloccanti** fino a che non si libera un posto eseguendo una lettura con successo
- Se viene costruito un canale sincrono di dimensione `0`, diventa un canale di tipo **rendezvous**: ogni operazione di lettura deve sovrapporsi temporalmente ad una di scrittura
 - Le restanti operazioni offerte da `SyncSender<T>` hanno semantica simile alle corrispondenti offerte da `Sender<T>`

Canali sincroni

```
use std::time::Duration;
use std::sync::mpsc::sync_channel;
use std::thread;

fn main() {

    let (sender, receiver) = sync_channel(1) ;

    thread::spawn(move || {
        sender.send(1).unwrap();
        println!("Sent: 1");
        sender.send(2).unwrap();
        println!("Sent: 2");
    });

    println!("{}", receiver.recv().unwrap());
    thread::sleep(Duration::from_secs(3));
    println!("{}", receiver.recv().unwrap());
}
```



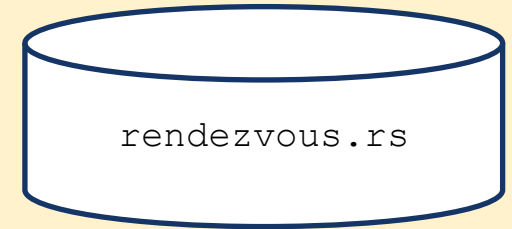
```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (sender, receiver) = mpsc::sync_channel(0);

    // Questo thread tenta di inviare un valore al canale, ma essendo la capacità 0,
    // si bloccherà fino a quando il valore non verrà letto dal receiver.
    thread::spawn(move || {
        sender.send("Sto trasmettendo un messaggio e ci aspettiamo
                    all'appuntamento").unwrap();

        println!("Valore inviato.");
    });

    // Il receiver legge il valore dal canale.
    let received_value = receiver.recv().unwrap();
    println!("Valore ricevuto: {}", received_value);
}
```



La libreria Crossbeam

- Variaggheta libreria ben documentata e attivamente mantenuta che offre una serie di costrutti a supporto dell'elaborazione concorrente
 - **Costrutti atomici** - la struct `crossbeam::atomic::AtomicCell<T>` estende il concetto di mutabilità interna offerto da `Cell<T>` a contesti *multithread*, appoggiandosi a **primitive atomiche** là dove possibile, oppure ricorrendo all'uso di un lock interno per strutture dati più articolate
 - **Strutture dati concorrenti** - le struct del crate `crossbeam::deque` (**Injector**, **Stealer** e **Worker**) offrono un meccanismo strutturato per la creazione di schedulatori basati sul furto di attività da eseguire; le struct `crossbeam::queue::{ArrayQueue, SegQueue}` implementano code di messaggi (limitate o illimitate) basate sul paradigma *multiple-producer-multiple-consumer*
 - **Canali MPMC** - le funzioni `crossbeam::channel::{bounded(...), unbounded()}` creano canali unidirezionali con capacità limitata o illimitata basati sul paradigma MPMC i cui estremi possono essere condivisi per semplice clonazione; le funzioni `crossbeam::channel::{after(...), tick(...)}` creano il solo estremo di ricezione che consegnerà un messaggio dopo il tempo indicato o periodicamente

Only for Rustaceans



`crossbeam_utils::atomic::AtomicCell`

- Primitiva utile quando si ha bisogno di scambiare valori tra thread in modo atomico.
- A differenza di `std::sync::atomic::Atomic*`, che lavora solo su tipi interi o booleani,
- `AtomicCell` può contenere qualsiasi tipo che implementi il tratto `Send`.

```
use crossbeam_utils::atomic::AtomicCell;
use std::sync::Arc;
use std::thread;
use std::time::Duration;
```

Only for Rustaceans



```
fn main() {
    // Creiamo un Arc<AtomicCell<Option<String>>>
    //   - Arc: per condividere la AtomicCell tra i thread.
    //   - AtomicCell: per garantire operazioni atomiche sulla cella.
    //   - Option<String>: per indicare la presenza o l'assenza di un messaggio.
    //   Inizialmente è None.
    let message_cell = Arc::new(AtomicCell::new(None));

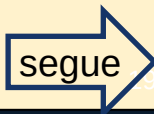
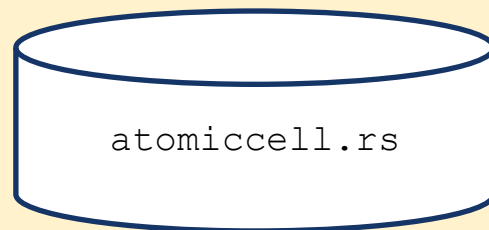
    let producer_cell = Arc::clone(&message_cell);

    let producer_handle = thread::spawn(move || {
        println!("[Produttore] Inizio...");
        thread::sleep(Duration::from_millis(500)); // Simula del lavoro

        let message = String::from("Hello from the producer!");

        // swap restituisce il vecchio valore (qui None).
        let old_value = producer_cell.swap(Some(message));
        println!("[Produttore] Messaggio inserito nella cella. Vecchio valore: {:?}", old_value);

        thread::sleep(Duration::from_millis(500)); // Simula altro lavoro
        println!("[Produttore] Fine.");
    });
};
```



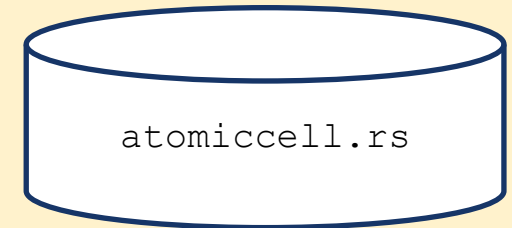
```

let consumer_cell = Arc::clone(&message_cell);
let consumer_handle = thread::spawn(move || {
    let mut received_message: Option<String> = None;

    loop {
        println!("[Consumatore] In attesa del messaggio...");
        thread::sleep(Duration::from_millis(100)); // Attendiamo brevemente
        // Atomicamente proviamo a prendere il valore dalla cella.
        // take imposta la cella a None e restituisce il valore precedente.
        if let Some(msg) = consumer_cell.take() {
            println!("[Consumatore] Messaggio ricevuto: '{}'", msg);
            received_message = Some(msg);
            break; // Usciamo dal loop una volta ricevuto
        }
    }

    println!("[Consumatore] Fine.");
});
producer_handle.join().unwrap();
consumer_handle.join().unwrap();
println!("\n[Main] Programma terminato.");
}

```



crossbeam_channel

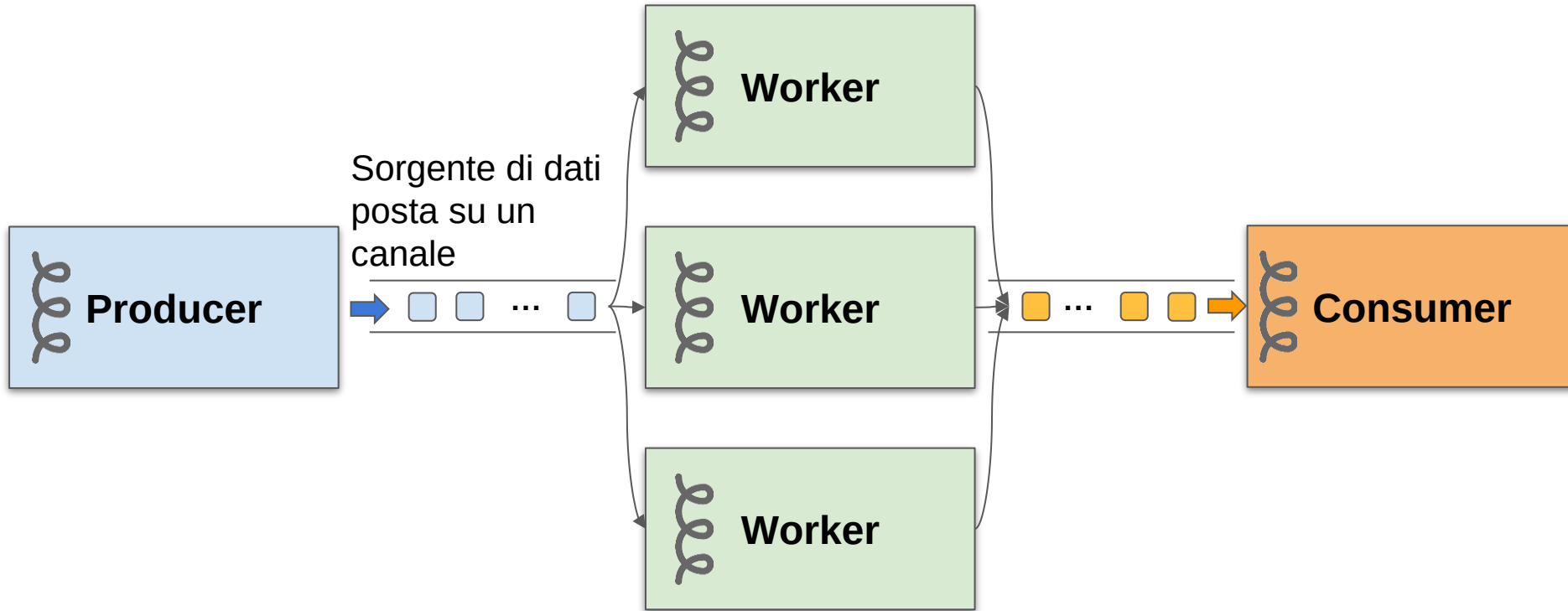
- Il paradigma MultipleProducersMultipleConsumers offre un meccanismo potente per l'implementazione di pattern concorrenti in Rust
- Il messaggio è sempre consumato una volta sola, se ci sono 2 thread che cercano di leggerlo, il messaggio viene letto solo da 1 dei 2
- Va bene, dunque, se non mi interessa sapere chi andrà effettivamente a leggere il messaggio.

crossbeam_channel

- **Fan-out / Fan-in** - permette di distribuire attività a più thread indipendenti e raccogliere i risultati prodotti in un singolo punto; usa una coppia di canali per distribuire e raccogliere i dati
- **Pipeline** - crea una serie di fasi di lavorazione, ciascuna delle quali è eseguita da un singolo thread e utilizza un canale per inoltrare i semi-lavorati tra due fasi successive
- **Producer / consumer** - consente ad uno o più thread produttori di generare valori che saranno elaborati dal primo thread consumatore disponibile; usa un singolo canale per la comunicazione

Fan-Out / Fan-In

I dati, prima di essere usati, devono essere elaborati



```

use std::thread;
use crossbeam_channel::{bounded, Receiver, Sender};
fn worker(id: usize, rx: Receiver<i32>, tx: Sender<String>) {
    thread::sleep(Duration::from_millis(200));
    while let Ok(value) = rx.recv() {
        tx.send(format!("W{} ({})", id, value)).unwrap();
    }
}

fn main() {
    let (tx_input, rx_input) = bounded:<i32>(10); // canale PW (Producer - Worker)
    let (tx_output, rx_output) = bounded:<String>(10); // canale WC (Worker - Consumer)
    let mut worker_handles = Vec::new();
    for i in 0..3 {
        let rx = rx_input.clone(); // clono il ricevitore del canale PW
        let tx = tx_output.clone(); // clono il trasmettitore del canale WC
        worker_handles.push( thread::spawn(move || worker(i, rx, tx)) );
    }
    for i in 1..=10
    {
        thread::sleep(Duration::from_millis(100));
        tx_input.send(i).unwrap(); // main thread trasmette sul canale PW
    }
    drop(tx_input);
    drop(tx_output);

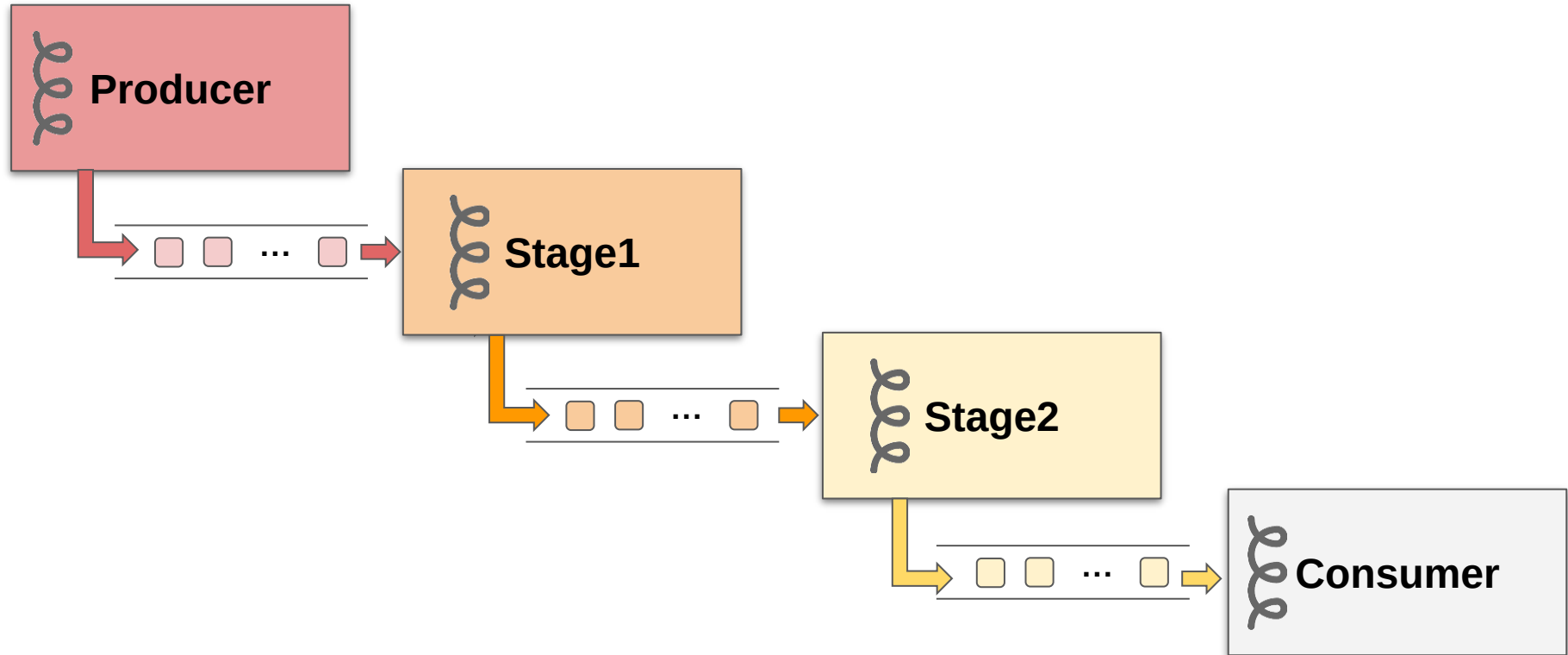
    while let Ok(result) = rx_output.recv() { // main thread legge dal canale WC
        println!("Received result: {}", result);
    }
    for handle in worker_handles { handle.join().unwrap(); }
}

```

fanoutfanin.rs

Pipeline

Sequenza di stadi che processano i dati



```

use std::thread;
use crossbeam_channel::{bounded, Receiver, Sender};
fn stage_one(rx: Receiver<i32>, tx: Sender<String>) {
    while let Ok(value) = rx.recv() {
        tx.send(format!("S1({})", value)).unwrap();
    }
}
fn stage_two(rx: Receiver<String>, tx: Sender<String>) {
    while let Ok(value) = rx.recv() {
        tx.send(format!("S2( {} )", value)).unwrap();
    }
}
fn main() {
    let (tx_input, rx_input) = bounded::<i32>(10); // Canale PS1 (tra Producer e Stage 1)
    let (tx_stage_one, rx_stage_one) = bounded::<String>(10); // Canale S1S2 (tra gli Stage)
    let (tx_output, rx_output) = bounded::<String>(10); // Canale S2C (tra Stage 2 e Consumer)

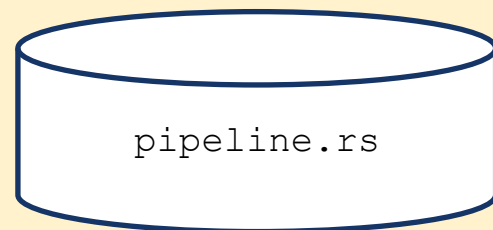
    let stage_one_handle = thread::spawn(move || stage_one(rx_input, tx_stage_one));
    let stage_two_handle = thread::spawn(move || stage_two(rx_stage_one, tx_output));

    for i in 1..=10 { tx_input.send(i).unwrap(); }
    drop(tx_input);

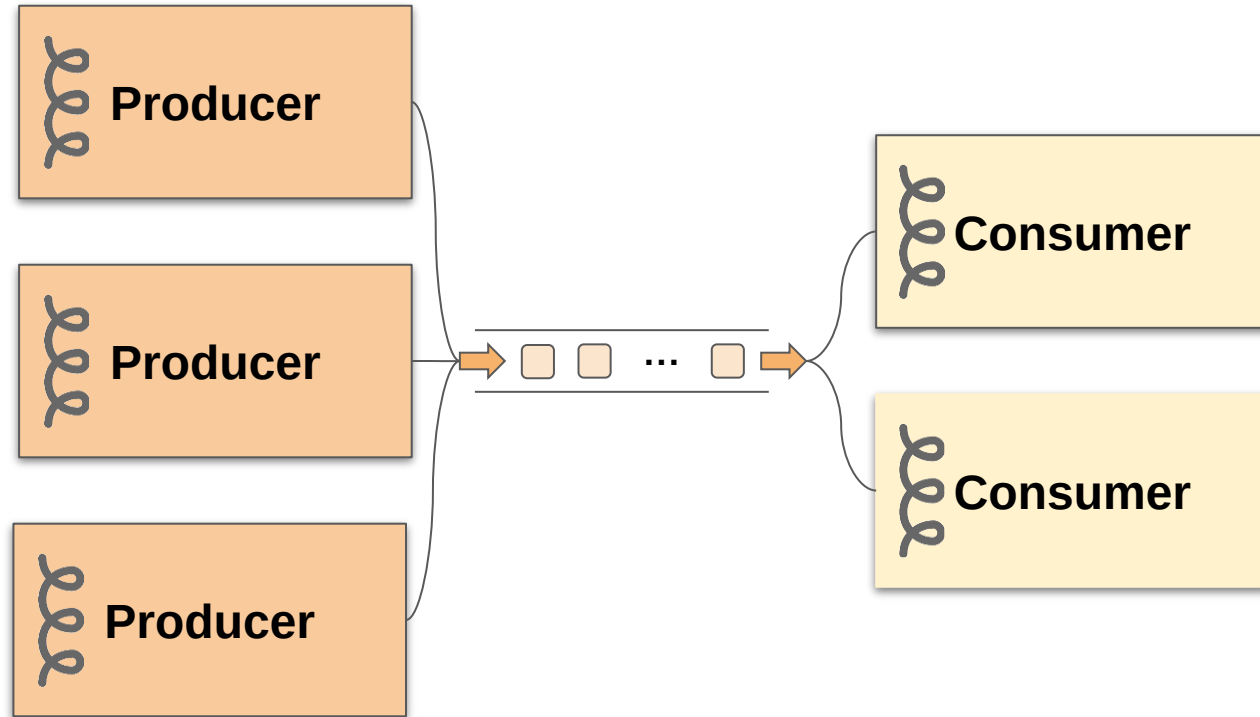
    while let Ok(result) = rx_output.recv() { println!("Received result: {}", result); }

    stage_one_handle.join().unwrap();
    stage_two_handle.join().unwrap();
}

```



Producer/consumer



```

use std::thread;
use crossbeam_channel::{bounded, Receiver, Sender};

fn producer(id: usize, tx: Sender<(usize,i32)>) {
    for i in 1..=5 { tx.send((id,i)).unwrap(); }
}

fn consumer(id: usize, rx: Receiver<(usize,i32)>) {
    thread::sleep(Duration::from_millis(150));
    while let Ok((sender_id, val)) = rx.recv() {
        println!("Consumer {} received {} from {}", id, val, sender_id);
    }
}

fn main() {
    let (tx, rx) = bounded::<(usize,i32)>(10);

    let mut handles = Vec::new();
    for i in 0..3 {
        let tx = tx.clone();
        handles.push( thread::spawn(move || producer(i, tx)) );
    }
    for i in 0..2 {
        let rx = rx.clone();
        handles.push(thread::spawn(move || consumer(i, rx)));
    }
    drop(tx);
    for handle in handles { handle.join().unwrap(); }
}

```



producerconsumer.rs

```

fn main() {
    let (tx, rx) = unbounded();

    let num_producers = 5;
    let mut producer_handles = vec![];

    for i in 0..num_producers {
        let tx_clone = tx.clone(); // Ogni produttore ottiene una copia del mittente (tx)
        let producer_id = i;

        let handle = thread::spawn(move || {
            for j in 0..3 { // Ogni produttore invia 3 messaggi
                let message = format!("Messaggio {} dal produttore {}", j, producer_id);
                println!("[Produttore {}] Invio: '{}'", producer_id, message);
                if let Err(e) = tx_clone.send(message) {
                    break; // Esci dal loop se il ricevitore è stato droppato
                }
                thread::sleep(Duration::from_millis(50 * (producer_id as u64 + 1))); // Simula lavoro
            }
            println!("[Produttore {}] Finito di inviare.", producer_id);
        });
        producer_handles.push(handle);
    }

    let consumer_handle = thread::spawn(move || {
        println!("[Consumatore] In attesa di messaggi...");
        // Il consumatore riceve messaggi finché il canale è aperto e ci sono messaggi da ricevere.
        // rx.iter() blocca finché non riceve un messaggio o il canale si chiude.
        for message in rx.iter() {
            println!("[Consumatore] Ricevuto: '{}'", message);
            thread::sleep(Duration::from_millis(100)); // Simula elaborazione
        }
        println!("[Consumatore] Canale chiuso. Finito di ricevere.");
    });

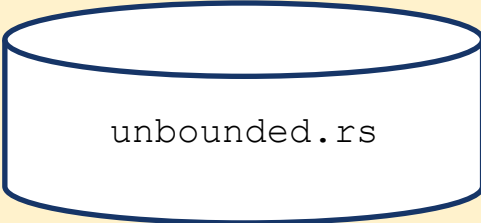
    for handle in producer_handles {
        handle.join().expect("Il thread produttore non è terminato correttamente");
    }

    // Quando tutti i mittenti (incluso l'originale in main) sono stati droppati,
    // il canale si chiude, e rx.iter() nel consumatore terminerà.
    // Se non droppassimo tx qui, il consumatore rimarrebbe in attesa indefinitamente
    // anche dopo che tutti i produttori hanno finito.
    drop(tx);
    println!("[Main] Tutti i produttori hanno finito e il mittente principale è stato droppato.");

    consumer_handle.join().expect("Il thread consumatore non è terminato correttamente");

    println!("[Main] Programma terminato.");
}

```



unbounded.rs

Only for Rustaceans



Canali `after` e `tick`

- La funzione **`after(duration)`** crea un ricevitore che invia un singolo messaggio (un `()`) dopo una certa durata di tempo (`duration`)
- La funzione **`tick(duration)`** crea un ricevitore che invia un messaggio (un `()`) ad una determinata frequenza di trasmissione (tempo pari a `duration`)..

Only for Rustaceans



```
use crossbeam_channel::select;
use crossbeam::channel;
use std::thread;
use std::time::{Duration, Instant};
fn main() {
```

```
    let (s, r) = channel::bounded(1);
    let after = channel::after(Duration::from_secs(5)); // Timeout dopo 5 secondi
```

```
    thread::spawn(move || {
        for i in 0..10 {
            // Simula un ciclo di operazioni di calcolo intensivo
            let start = Instant::now();
            while start.elapsed() < Duration::from_secs(i) {
                // Fa qualcosa di computazionalmente intenso
            }
            let _ = s.send("Operazione completata").unwrap();
        }
    });
```

```
    loop {
        select! {
            recv(after) -> _ => {
                println!("Timeout! Operazione non completata"); // Timeout
                break;
            }
            recv(r) -> msg => println!("{}", msg.unwrap()), // Riceve il messaggio
        }
    }
}
```

select! è un costrutto fornito dalla crate crossbeam che permette di ascoltare più canali contemporaneamente e di eseguire azioni basate su quale canale riceve un valore per primo.



```

use crossbeam_channel::{tick, after, select};
use std::thread;
use std::time::Duration;

fn main() {
    let heartbeat_rx = tick(Duration::from_millis(500));

    let timeout_rx = after(Duration::from_secs(2)); // Canale per il timeout: invia un messaggio dopo 2 secondi

    // Canale per la segnalazione di completamento del processo
    let (process_completion_tx, process_completion_rx) = crossbeam_channel::bounded(0);

    let process_handle = thread::spawn(move || {
        println!("[Processo] Avviato...");
        // Simula un lavoro che potrebbe finire in un tempo variabile
        let work_duration_millis = (rand::random::lt(u64)() % 4500) + 500; // Tra 0.5s e 5s
        thread::sleep(Duration::from_millis(work_duration_millis));
        println!("[Processo] Lavoro completato in {} ms.", work_duration_millis);

        // Segnala il completamento al thread principale
        // send() blocca finché il ricevitore è pronto a ricevere (capacità 0)
        let _ = process_completion_tx.send();
        println!("[Processo] Segnalazione di completamento inviata.");
    });

    println!("[Monitor] In attesa di eventi...");
    loop {
        select! {
            recv(heartbeat_rx) -> _ => {
                println!("[Monitor] Segnale ricevuto! Il processo è ancora vivo.");
            },
            recv(process_completion_rx) -> _ => {
                println!("[Monitor] Processo completato con successo!");
                break; // Esci dal loop di monitoraggio
            },
            recv(timeout_rx) -> _ => {
                println!("[Monitor] ATTENZIONE: Timeout scaduto! Il processo non è finito in tempo.");
                let _ = process_completion_rx.recv();
                break; // Esci dal loop di monitoraggio
            },
        }
    }

    process_handle.join().expect("Il thread del processo non è terminato correttamente");
    println!("[Main] Programma terminato.");
}

```



aftertick.rs

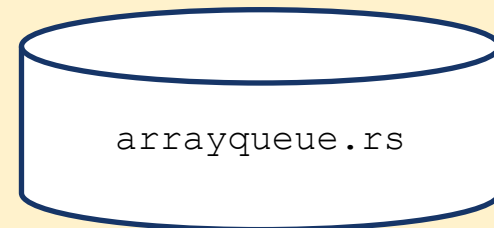


crossbeam::queue

- Modulo della libreria Crossbeam che fornisce 2 tipi di code che permettono accessi concorrenti (thread-safe) in modalità MPMC
 - `SegQueue<T>`: coda non bloccante FIFO che offre i seguenti principali metodi:
 - `push(item)`: inserisce un elemento in coda
 - `pop()`: rimuove un elemento dalla testa della coda; ritorna `Err` se vuota
 - `is_empty()`: verifica se la coda è vuota
 - `ArrayQueue<T>`: coda circolare a capacità fissa
 - `push(item) -> Result <(), T>`: inserisce un elemento nella coda e ritorna `Err` se la coda è piena (non blocca)
 - `pop() -> Option<T>`: rimuove un elemento dalla testa della coda; ritorna `None` se vuota (non blocca)
 - `is_empty()`: verifica se la coda è vuota
 - `len()`: ritorna il numero dei elementi presenti.



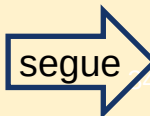
Only for Rusteceans



```
#[derive(Debug, Clone)]
enum Work {
    Task(u32),
    Stop, // Un messaggio speciale per indicare ai worker di fermarsi
}

fn main() {
    let queue = Arc::new(ArrayQueue::new(5));
    let num_tasks_to_generate = 15;
    println!("[Main] Creata ArrayQueue con capacità: {}", queue.capacity());
    let num_workers = 3; // --- Consumatori (Workers che elaborano i compiti) ---
    let mut worker_handles = vec![];

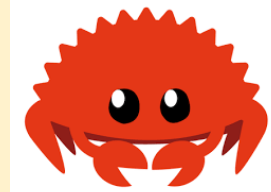
    for i in 0..num_workers {
        let q_clone = Arc::clone(&queue);
        let worker_id = i;
        worker_handles.push(thread::spawn(move || {
            println!("[Worker {}] In attesa di compiti...", worker_id);
            loop {
                match q_clone.pop() { // Tenta di prendere un compito dalla coda
                    Some(Work::Task(task_id)) => {
                        println!("[Worker {}] Elaboro Task {}", worker_id, task_id);
                        thread::sleep(Duration::from_millis(200)); // Simula lavoro
                    },
                    Some(Work::Stop) => {
                        println!("[Worker {}] Ricevuto segnale di STOP. Terminazione.", worker_id);
                        break; // Esci dal loop e termina il worker
                    },
                    None => {
                        // Coda vuota, aspetta un po' prima di riprovare per non sprecare CPU
                        thread::sleep(Duration::from_millis(10));
                    }
                }
            }
        }));
    }
}
```



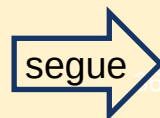
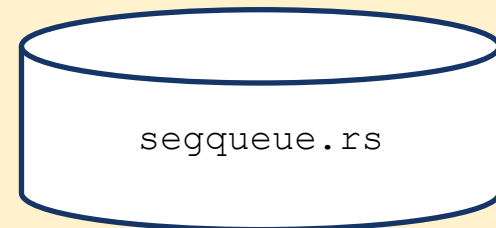


Only for Rusteceans

```
// --- Thread principale come produttore per semplicità
for i in 0..num_tasks_to_generate {
    let task = Work::Task(i);
    print!("[Main] Tentativo di invio: {:?}...", task);
    loop {
        // push restituisce Ok(()) se riesce, Err(Work) se la coda è piena
        match queue.push(task.clone()) {
            Ok(_) => {
                println!("Inviato.");
                break; // Task inviato, esci dal loop interno
            },
            Err(e) => {
                println!("Coda piena. Aspetto... (Non inviato: {:?})", e);
                thread::sleep(Duration::from_millis(100)); // Coda piena, attendi e riprova
            }
        }
    }
    thread::sleep(Duration::from_millis(50)); // Simula tempo tra le generazioni di task
}
println!("\n[Main] Tutti i compiti iniziali inviati. Invio segnale di STOP ai worker.");
for _ in 0..num_workers {
    // Se la coda è piena, questo loop si bloccherà finché non c'è spazio.
    loop {
        if queue.push(Work::Stop).is_ok() {
            break;
        }
        thread::sleep(Duration::from_millis(10));
    }
}
for handle in worker_handles {
    handle.join().unwrap();
}
}
```



Only for Rusteceans



```
#[derive(Debug, Clone)]
enum Event {
    Data(String),
    Shutdown, // Segnale per terminare il consumatore
}
```

```
fn main() {
    // Crea una SegQueue (illimitata)
    let queue = Arc::new(SegQueue::new());
    println!("[Main] SegQueue creata.");
```

```
    // --- Produttori ---
    let num_producers = 3;
    let mut producer_handles = vec![];
```

```
    for i in 0..num_producers {
        let q_clone = Arc::clone(&queue);
        let producer_id = i;
```

```
        producer_handles.push(thread::spawn(move || {
            println!("[Produttore {}] Avviato.", producer_id);
            for j in 0..5 { // Ogni produttore invia 5 eventi
                let event_data = format!("Dato_{}_{}", producer_id, j);
                let event = Event::Data(event_data);
```

```
                println!("[Produttore {}] Invio: {:?}", producer_id, event);
                q_clone.push(event); // push() non fallisce mai su SegQueue
                thread::sleep(Duration::from_millis(100 + (producer_id as u64 * 50))); // Simula generazione
```

irregolare

```
            }
            println!("[Produttore {}] Finito di inviare.", producer_id);
        }));
    }
}
```



Only for Rusteceans

```
// --- Consumatore ---
let q_consumer = Arc::clone(&queue);
let consumer_handle = thread::spawn(move || {
    println!("[Consumatore] In attesa di eventi...");
    let mut received_count = 0;
    loop {
        // pop() non blocca, restituisce None se la coda è vuota.
        match q_consumer.pop() {
            Some(Event::Data(data)) => {
                println!("[Consumatore] Elaboro: {}", data);
                received_count += 1;
                thread::sleep(Duration::from_millis(70)); // Simula elaborazione
            },
            Some(Event::Shutdown) => {
                println!("[Consumatore] Ricevuto segnale di SHUTDOWN. Terminazione.");
                break; // Esci dal loop
            },
            None => {
                // Coda vuota, aspetta un po' per non sprecare CPU in un busy-loop
                thread::sleep(Duration::from_millis(5));
            }
        }
    }
});
println!("[Consumatore] Terminazione. Elaborati {} eventi.", received_count);

// --- Il thread Main aspetta i produttori e poi invia i segnali di shutdown ---
for handle in producer_handles {
    handle.join().unwrap();
}
println!("\n[Main] Tutti i produttori hanno finito di inviare.");

// Dopo che tutti i produttori hanno finito, invia i segnali di shutdown al consumatore.
println!("[Main] Invio segnale di SHUTDOWN al consumatore.");
queue.push(Event::Shutdown); // push() è non bloccante e non fallisce.

consumer_handle.join().unwrap();
}
```



crossbeam::deque

- Il "work stealing" è una tecnica di bilanciamento del carico utilizzata in sistemi di calcolo parallelo per migliorare l'efficienza e l'utilizzo dei processori.
- Il work stealing è un metodo in cui i thread che rimangono inattivi rubano lavoro dai thread che sono ancora occupati. Questo meccanismo aiuta a bilanciare il carico di lavoro tra i thread, assicurando che tutti i processori disponibili siano utilizzati in modo efficiente
- Come funziona:
 - Ogni thread riceve una coda di lavoro contenente task ed esegue i task dalla propria coda.
 - Se un thread esaurisce i task nella propria coda, diventa inattivo e tenta di rubare task dalle code di altri thread
 - Un thread inattivo sceglie casualmente un altro thread e cerca di rubare un task dalla coda di quest'ultimo. Il furto di solito avviene dalla coda opposta rispetto alla direzione normale di estrazione del task, per minimizzare il conflitto (es. FIFO per il thread proprietario e LIFO per il ladro).
- Il `crate deque` permette di creare una serie di code:
 - `Worker<T>`, una coda deque (double-ended queue) locale ad un thread
 - `Stealer<T>`, una handle che consente ad altri thread di rubare lavoro dalla parte iniziale della coda `Worker`
 - `Injector<T>`, una coda globale condivisa usata per iniettare nuovi task, accessibile da tutti i thread.

```

use crossbeam::deque::{Injector, Steal, Stealer, Worker};
use std::sync::Arc;
use std::thread;

```

Only for Rustaceans



```

fn main() {
    let injector = Arc::new(Injector::new()); // Coda globale condivisa
    let worker = Worker::new_fifo(); // Coda locale del main thread
    let stealer = worker.stealer(); // Stealer per gli altri thread

    for i in 1..=3 { // Inserisci alcuni task nel Worker del main thread
        let task = format!("task {}", i);
        worker.push(task);
    }

    // Inserisci altri task nell'Injector globale
    for i in 4..=6 {
        let task = format!("task {}", i);
        injector.push(task);
    }

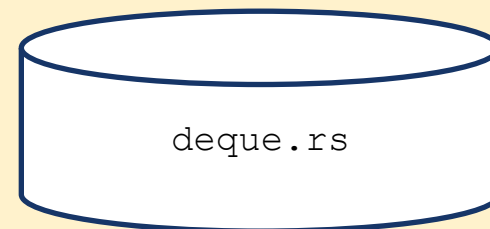
    let mut handles = vec![];

    for id in 0..2 {
        let injector = injector.clone();
        let stealer = stealer.clone(); // ogni thread riceve una copia

        let handle = thread::spawn(move || {
            loop {
                let task = match stealer.steal().or_else(|| injector.steal()) {
                    Steal::Success(task) => task,
                    Steal::Retry => continue,
                    Steal::Empty => break,
                };
                println!("[Ladro {id}] Eseguito: {task}");
            }
            println!("[Ladro {id}] Nessun altro task. Fine.");
        });
        handles.push(handle);
    }
    // Il main thread esegue i task locali dalla sua coda Worker
    while let Some(task) = worker.pop() {
        println!("[Main] Eseguito: {task}");
    }
    println!("[Main] Coda vuota. Attendo i ladri.");

    for handle in handles {
        handle.join().unwrap();
    }
}

```



Libreria Rayon

Only for Rustaceans



- Rayon è una libreria di Rust che semplifica enormemente la programmazione parallela, in particolare la parallelizzazione di dati (data parallelism)
- Uno degli strumenti che offre per il parallelismo è la funzione `join()`, che permette di eseguire due task in parallelo e aspettare che entrambi finiscano prima di procedere.
- La funzione `join` prende due chiusure e le esegue in parallelo. Attende che entrambe finiscano e restituisce una tupla con i risultati di entrambe le chiusure.
- `join` fornisce un modo semplice per eseguire due task in parallelo senza dover gestire esplicitamente i thread
- Rayon ottimizza automaticamente l'uso delle risorse hardware, rendendo l'esecuzione parallela efficiente.


```
extern crate rayon;

use rayon::prelude::*;
use rayon::join;

fn main() {
    // Definiamo due compiti che eseguiamo in parallelo
    let task1 = || {
        println!("Esecuzione del task 1");
        // Simuliamo un lavoro computazionale
        let sum: i32 = (1..=25_000).sum();
        println!("Il risultato del task 1 è: {}", sum);
        sum
    };

    let task2 = || {
        println!("Esecuzione del task 2");
        // Simuliamo un lavoro computazionale diverso
        let product: i32 = (1..=12).product();
        println!("Il risultato del task 2 è: {}", product);
        product
    };

    // Eseguiamo i due compiti in parallelo
    let (result1, result2) = join(task1, task2);

    // Utilizziamo i risultati
    println!("Il risultato combinato è: {}", result1 + result2);
}
```



- Rayon usa un modello di fork-join: se ha un compito grande, lo divide in due compiti più piccoli (fork). Questi due compiti possono essere eseguiti in parallelo. Se un compito è ancora troppo grande, viene diviso di nuovo, e così via. *
- Pool di Thread: Rayon gestisce un suo "pool di thread" (un gruppo di thread pronti a lavorare). Quando un compito viene diviso, una parte viene eseguita dal thread corrente e l'altra parte può essere messa nella coda di lavoro del pool per essere presa da un altro thread. Il metodo `par_iter()` è un iteratore che distribuisce automaticamente il lavoro su più thread, utilizzando il pool di thread gestito da Rayon
- Work-Stealing: Questo è il motore nascosto di Rayon e usa `crossbeam::deque`. Se un thread finisce i suoi compiti e altri thread sono ancora occupati, il thread va a "rubare" compiti dalla coda di un altro thread. Questo assicura che tutti i core siano sempre occupati, bilanciando il carico di lavoro in modo dinamico e molto efficiente.

```
use rayon::prelude::*; // Importa i tratti necessari per gli iteratori paralleli
```

```
fn main() {  
    let numbers: Vec<i64> = (0..1_000_000).collect();  
  
    // Calcolo sequenziale  
    let mut sum_sequential:i64 = 0;  
    let start_seq = std::time::Instant::now();  
    for n in &numbers {  
        sum_sequential += n;  
    }  
    let duration_seq = start_seq.elapsed();  
    println!("Somma sequenziale: {} (in {:?})", sum_sequential, duration_seq);  
  
    // Calcolo parallelo con Rayon  
    let start_par = std::time::Instant::now();  
    let sum_parallel: i64 = numbers.par_iter().sum();  
    // .par_iter() trasforma l'iteratore in parallelo  
    let duration_par = start_par.elapsed();  
    println!("Somma parallela: {} (in {:?})", sum_parallel, duration_par);  
}
```



```

use rayon::prelude::*;
use rayon::ThreadPoolBuilder;

fn main() {
    let pool = ThreadPoolBuilder::new().num_threads(4)
        .stack_size(100_000)
        .thread_name(|index| format!("my number - {}", index))
        .build().unwrap();

    pool.install(|| {
        let v: Vec<i32> = (1..20_000).collect();

        let sum: i32 = v.par_iter()
            .map(|&x| x * 2)
            .sum();

        println!("Somma: {}", sum);
    });
}

```



- Si può creare e configurare un pool di thread personalizzato e usarlo per eseguire operazioni parallele
 - build() costruisce il pool di thread con le impostazioni specificate
 - install() esegue un blocco di codice nel contesto del pool di thread

Glossario

- Produttore: thread che scrive un messaggio nel canale
- Consumatore: thread che legge il messaggio dal canale
- Unbound: canale di dimensione illimitata
- Backpressure: meccanismo per cui un componente più lento in un sistema segnala ai componenti più veloci a monte di rallentare la loro produzione di dati. I canali bounded limitano la capacità del canale e quando il canale è pieno la trasmissione è bloccata.